OpsPilot — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and where each lives

Keep one job per file (the same layout as any `src/` project):

- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py` — settings, named errors, one logging setup. **Log to stderr** — an MCP client can carry protocol traffic on stdout.
- `servers/status_server.py`, `servers/runbook_server.py` — two tiny `MCPServer` services so the whole thing runs locally. They're launched as subprocesses, never imported.
- `src/server_pool.py` — `MCPServerPool`: connect to every configured service, discover its tools (`list_tools()`), namespace each (`service__tool`), route calls back. The heart of it.
- `src/tool_bridge.py` — `mcp_tools_to_openai_schema()` and `extract_text()`.
- `src/models/llm.py` — the one file that talks to OpenAI (`ask_model`).
- `src/agent.py` — `run_agent()`, the ReAct loop.
- `src/servers_config.py` — the list of services, as data.
- `main.py` — connect, discover, run one question, print each step.

A build order that works: one service discovered and printed → a loop over it → add the second service and prove routing → add the timeout/skip and the mid-call catch. Your call.

Hint 2 — discovery, and the `input_schema` gotcha

The MCP client pattern (Doc06):

```
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client

params = StdioServerParameters(command="python",
                               args=["servers/status_server.py"])
async with stdio_client(params) as (read, write):
    async with ClientSession(read, write) as session:
        await session.initialize()                # required handshake
        listed = await session.list_tools()
```

```
        for tool in listed.tools:
            print(tool.name, tool.description, tool.input_schema)
```

- A discovered tool's schema is `tool.input_schema` (**snake_case**) in the current `mcp` SDK — `tool.inputSchema` raises `AttributeError`.
- `await session.call_tool(name, args)` returns a `CallToolResult` whose `.content` is a list of blocks (`TextContent` , with `.text`) — not a string. Join the `.text` s.
- Namespace every tool as `f"{service}__{tool.name}"` and keep a routing dict `{"status__get_service_status": ("status", "get_service_status")}`. Use `__` , not `.` — OpenAI function names allow only letters, digits, `_` , `-` .

Hint 3 — the loop, and the schema bridge

The bridge is mostly a rename, because MCP's `input_schema` is already JSON Schema:

```
{
  "type": "function",
  "function": {
    "name": namespaced_name,         # "status__get_service_status"
    "description": tool.description or "",
    "parameters": tool.input_schema, # passed straight through
  },
}
```

The loop is the standard tool-calling loop (no `create_agent`):

```
for step in range(max_iterations):
    reply = ask_model(client, messages, pool.openai_tools())
    if not reply.tool_calls: return AgentResult(reply.content, steps)
    append the assistant turn (carry its tool_calls back as dicts!)
    for call in reply.tool_calls:
        obs = await pool.call_tool(call.function.name,
                                   json.loads(call.function.arguments))
        append {"role":"tool","tool_call_id":call.id,"content":obs}
raise MaxIterationsExceeded(...)
```

The one subtlety: the assistant message that *made* the tool calls must carry those `tool_calls` back (as plain dicts), or the follow-up `tool` messages have nothing to attach to and the next API call is rejected.

Hint 4 — resilience, and the async trap that will cost you an hour

Three guards on the pool:

- **A service that won't connect** — `try/except` around each service in `connect_all`; log and `continue`, so the others still connect.
- **A service that hangs starting** — wrap `session.initialize()` in `asyncio.wait_for(..., timeout=...)`.
- **A mid-incident drop** — `try/except` around the `call_tool` dispatch; return a clear `"Error: …"` string as the observation.

The trap: do **not** write `await asyncio.wait_for(self._connect_one(service), ...)`. `wait_for` runs its coroutine in a *new task*, so the `stdio_client` / `ClientSession` contexts get entered in that task — and closing them later in the main task raises `RuntimeError: Attempted to exit cancel scope in a different task`. Enter the contexts in the main task; wrap only `initialize()` in the timeout.

Still stuck? The whole thing, every file, is in the complete solution.